Plano do estudo: testes automatizados de áudio no tapSynth

Resumo

O estudo mede quais estratégias de teste automatizado detectam quais tipos de defeito de áudio em um plugin VST3, com quantos alarmes falsos, e compara essas estratégias com o pluginval, o validador genérico mais usado por desenvolvedores JUCE.

O sistema testado é o tapSynth. Os defeitos são injetados em duas origens de código: o código do próprio projeto e as classes de DSP do JUCE que ele usa. Isso permite separar o efeito do autor (código que nós escrevemos) do código de terceiros.

Contribuição: evidência empírica sobre qual teste pega qual defeito de áudio, um conjunto de defeitos e uma infraestrutura reprodutíveis que outros podem reutilizar, e um guia prático para desenvolvedores.

Entregas:

- Artigo do PFC I com os resultados e a discussão.
- Repositório público e reproduzível: testes, código de análise de áudio, defeitos, scripts e pipeline.
- Guia de recomendação: que testes escrever, com que tolerância, e o que o pluginval cobre ou não.

Perguntas de pesquisa e hipóteses

Pergunta geral: como diferentes estratégias de teste automatizado de áudio se comparam, em detecção de defeitos e em alarmes falsos, na verificação de um plugin VST3?

Pergunta	Hipótese
QP1. Que estratégia detecta que categoria de defeito, e com quantos alarmes falsos?	Cada estratégia se especializa: espectral em altura e filtro, temporal em envelope, sanidade em problemas numéricos e de nível.
QP2. Como as estratégias de áudio se comparam ao pluginval?	O pluginval detecta defeitos de estado, host e travamento, mas poucos defeitos de conteúdo sonoro.

Pergunta	Hipótese
QP3. Como a tolerância afeta o compromisso entre detecção e alarmes falsos?	O golden master detecta mais defeitos que as outras estratégias, mas com mais alarmes falsos, e o equilíbrio depende da tolerância.
QP4. O desempenho muda entre defeitos no código do projeto e no código do JUCE?	Não há diferença relevante; se houver, indica viés de autoria.

A comparação entre frameworks de C++ deixou de ser pergunta de pesquisa e virou uma análise secundária (seção "Comparação de frameworks").

Objeto de estudo

O sistema testado é o tapSynth: um plugin sintetizador em JUCE 8.0.9, compilado com CMake, com 5 vozes. Cada voz soma dois osciladores (seno, dente de serra ou quadrada, com FM), aplica ganho fixo, envelope ADSR de amplitude e um filtro de variáveis de estado com modulação por envelope e LFO. Depois da soma das vozes vêm um reverb e um medidor de nível.

Todos os testes tocam notas no `TapSynthAudioProcessor` real e analisam o som produzido. Os defeitos são injetados em duas origens de código:

Origem	O que entra	Por que importa
Código do projeto	`OscData` (nota MIDI → frequência, transposição, FM), `SynthVoice` (soma da modulação e limite do cutoff), fórmulas das formas de onda, `MeterData`, `applyParametersFromJson`	É o código específico do plugin, onde ficam os erros de integração
Código do JUCE	`juce::ADSR`, `juce::dsp::Oscillator`, `juce::dsp::StateVariableTPTFilter`, `juce::dsp::Reverb`, `juce::dsp::Gain`	Código que não escrevemos e que milhares de plugins usam; reduz o viés de autoria

Origem do código: o tapSynth começou como o tutorial tapSynth do The Audio Programmer (Joshua Hodge, 2021; os cabeçalhos dos arquivos em `Source/Data/` mostram isso). Um colega o estendeu com FM, modulação do filtro, nova interface e o recurso de IA. Este trabalho contribui com os testes, os defeitos, a pipeline e a avaliação. As três partes devem ser creditadas no artigo e no README.

Fora do escopo: testes de interface gráfica, a chamada à API do Gemini e outros formatos (AU, AAX). O recurso de IA entra apenas de forma indireta: a estratégia de sanidade aplica patches aleatórios pelo mesmo caminho que as respostas da IA usam.

Abordagens de detecção

São cinco abordagens: quatro estratégias de teste de áudio escritas por nós e o pluginval como linha de base.

Abordagem	Pergunta que responde	Exemplos de verificação
Sanidade	O som está quebrado?	Sem NaN ou infinito; pico abaixo de um limite; sem silêncio inesperado; sem DC. Variante com patches aleatórios: N conjuntos de parâmetros aplicados por `applyParametersFromJson`, com as mesmas verificações
Espectral	As frequências estão certas?	Frequência fundamental de cada nota em 44,1, 48 e 96 kHz; atenuação do passa-baixa acima do cutoff; aliasing das ondas dente de serra e quadrada (teste de caracterização)
Temporal	O som evolui certo no tempo?	O ataque atinge o pico no tempo configurado; o release chega ao silêncio; a voz é liberada após o release
Golden master	O som mudou?	Áudio renderizado comparado com referências gravadas, em configurações padrão e extremas, com tolerância variada
pluginval (linha de base)	O plugin se comporta bem num host?	Carregamento, salvamento e restauração de estado, parâmetros aleatórios, chamadas de várias threads, taxas de amostragem e tamanhos de bloco variados

Regras do pluginval: roda sobre o VST3 compilado, não sobre o executável de testes. O nível de rigor (de 1 a 10) é fixado antes do experimento e informado no artigo.

Regra das referências do golden master: são gravadas uma única vez, a partir do código sem defeitos, e versionadas no repositório. Se o sintetizador tiver qualquer aleatoriedade, a semente é fixada nos testes.

Defeitos injetados e alterações inofensivas

Cada defeito é aplicado sozinho, um por vez, e todas as abordagens rodam contra ele.

Categorias de defeito (definidas antes de criar os defeitos):

Categoria	Exemplo
Altura/frequência	Taxa de amostragem fixa em 44,1 kHz no cálculo do oscilador
Nível/ganho	Ganho de saída dobrado
Tempo/envelope	O release nunca chega ao fim
Filtro	Cutoff em unidade errada; limite do cutoff removido
Numérico	Filtro instável na ressonância máxima, gerando NaN
Estado/host	Restauração de estado ignora um parâmetro; travamento com bloco de 1 amostra; alocação de memória no callback de áudio

A categoria estado/host existe para a comparação com o pluginval ser justa: se todos os defeitos fossem de conteúdo sonoro, o pluginval perderia por construção.

Fontes dos defeitos:

1. **Baseados em correções reais** de plugins JUCE open source: 10 a 15 patches, criados pelo colega (não por quem escreve os testes). Cada patch registra a correção real que o inspirou (link do commit ou issue).
2. **Mutantes do Mull no código do projeto:** gerados automaticamente.
3. **Mutantes do Mull nas classes de DSP do JUCE:** restritos aos arquivos das classes listadas no objeto de estudo.

Cada arquivo de defeito tem um cabeçalho com identificador, categoria, origem (projeto ou JUCE) e fonte. Mutantes que nenhuma abordagem detecta são classificados à parte, porque podem ser equivalentes (não mudam o comportamento).

Alterações inofensivas (medem alarmes falsos; qualquer falha nelas conta como alarme falso):

- refatoração sem mudança de comportamento;
- build Release em vez de Debug;
- outro sistema operacional (Windows, macOS);
- outro compilador.

Métricas e análise

Todas as métricas e tolerâncias são fixadas antes da execução, para que os resultados não dependam de escolhas feitas depois.

Métrica	Pergunta	Como é medida
Taxa de detecção	QP1, QP2	Defeitos detectados ÷ defeitos injetados, por abordagem e por categoria
Alarmes falsos	QP1, QP3	Número de alterações inofensivas que fizeram a abordagem falhar
Curva de tolerância	QP3	Detecção e alarmes falsos de cada estratégia em uma grade de tolerâncias definida antes
Detecção por origem	QP4	Taxa de detecção separada para defeitos no projeto e no JUCE
Custo de execução	Discussão	Tempo de execução por abordagem e linhas de código dos testes

Análise estatística: taxas apresentadas com intervalo de confiança de 95% (Wilson). Como todas as abordagens rodam sobre os mesmos defeitos, a comparação é pareada: teste Q de Cochran entre as cinco abordagens e testes de McNemar par a par, com correção para comparações múltiplas.

Apresentação dos resultados:

- matriz defeito × abordagem (detectou ou não);
- taxa de detecção por categoria × abordagem;
- alarmes falsos por abordagem;
- gráfico da curva de tolerância (detecção contra alarmes falsos).

Arquitetura técnica

Executável de testes. O `juce_add_plugin(tapSynth ...)` já gera o código compartilhado do plugin como uma biblioteca chamada `tapSynth`, que os wrappers VST3 e AU usam. Um executável `tapSynth_tests` pode ligar essa mesma biblioteca e o GoogleTest (via FetchContent). O `main` dos testes precisa de um `juce::ScopedJuceInitialiser_GUI`. Isso dispensa a extração de um `tapsynth_core`, mas deve ser confirmado na primeira tarefa da fase 0.

Renderização. Uma função auxiliar cria o `TapSynthAudioProcessor` sem editor, chama `prepareToPlay` com a taxa de amostragem e o tamanho de bloco desejados, define parâmetros pelo `apvts`, monta um `juce::MidiBuffer` com note-on e note-off e chama `processBlock` em laço, acumulando a saída.

Análise de áudio (compartilhada por todos os testes): `rms`, `peak`, `fundamentalHz` (FFT com `juce::dsp::FFT`), `bandEnergyDb`, `envelope`, `maxAbsDiff`, `loadWav` e `saveWav`.

Estrutura de pastas:

```
tapSynth/
├─ Source/                código do plugin (inalterado)
├─ tests/
|  ├─ common/             renderização e análise de áudio
|  ├─ sanity/  spectral/  temporal/  golden/
|  └─ references/         WAVs de referência do golden master
├─ experiment/
|  ├─ faults/             um patch por defeito (adicionado só depois do
congelamento dos testes)
|  ├─ harmless/           alterações inofensivas
|  ├─ run_experiment.py   aplica, compila, roda tudo, registra e desfaz
|  └─ results/            results.csv e relatórios XML
└─ .github/workflows/     build.yml + tests.yml + mutation.yml
```

JUCE local para os defeitos no JUCE. O CMakeLists.txt já aceita `-DTAPSYNTH_JUCE_DIR=/caminho/para/JUCE`. Clonar o JUCE 8.0.9 uma vez e aplicar os patches ou rodar o Mull nessa cópia, sem tocar na versão baixada pelo FetchContent.

Mull. Roda no Linux, compilando com uma versão do Clang compatível com o Mull. O escopo é limitado aos arquivos do projeto em `Source/` (sem `Source/UI/`) e aos arquivos das classes JUCE listadas. Várias dessas classes ficam em arquivos de cabeçalho: confirmar na fase 0 que o Mull gera mutantes nelas.

CI (GitHub Actions):

- `build.yml` (já existe): compila em Linux, Windows e macOS.
- `tests.yml`: roda os testes nos três sistemas e publica relatórios XML.
- `mutation.yml`: roda o Mull e o experimento completo no Linux, sob demanda (`workflow_dispatch`), porque é lento.

Script do experimento. Para cada defeito: aplicar o patch, compilar os testes e o VST3, rodar os testes e o pluginval, registrar o resultado e desfazer o patch. Cada linha do `results.csv` tem: `defect_id`, `origin`, `category`, `approach`, `tolerance`, `detected` (0 ou 1) e `runtime_s`.

Controle de viés e regras para o Claude Code

A maior ameaça à validade é quem escreve os testes conhecer os defeitos. Estas regras valem para todos, pessoas e IA:

1. **Testes congelados antes dos defeitos.** A suíte completa é escrita e marcada com a tag git `tests-frozen` antes de qualquer defeito existir. Depois disso, os testes não mudam.
2. **Defeitos realistas criados pelo colega**, em uma branch ou repositório separado, sem informar a quem escreve os testes o que são.
3. **O Claude Code que escreve testes não vê `experiment/faults/`.** A pasta só entra no repositório depois do congelamento.
4. **Tolerâncias, grade de tolerâncias e nível do pluginval fixados antes da execução**, registrados em commit.
5. **Uso de IA declarado** na seção de Materiais e Métodos: o que foi gerado com ajuda de IA e como foi revisado.
6. **Toda mudança revisada por uma pessoa antes do commit.**
7. **A chave da API do Gemini nunca entra no repositório.**

Trecho sugerido para o CLAUDE.md:

```
## Estudo de testes (TCC)

Este repositório é o objeto de um estudo sobre testes automatizados de áudio.
O plano completo está no doc "Plano do estudo: testes automatizados de áudio
no tapSynth".

- Testes ficam em tests/ e usam GoogleTest; o executável tapSynth_tests liga
a biblioteca tapSynth.
- Todo teste renderiza áudio pelo TapSynthAudioProcessor real (sem editor).
- Não leia, crie nem altere nada em experiment/faults/. Não escreva testes
pensando em defeitos específicos.
- Depois da tag tests-frozen, não altere arquivos em tests/.
- Tolerâncias ficam em tests/common/Tolerances.h e só mudam com aprovação
explícita.
- Nunca escreva uma chave de API em nenhum arquivo.
```

Plano de execução

As fases seguem esta ordem; a fase 0 elimina os riscos técnicos antes de qualquer trabalho grande.

Fase 0: viabilidade (1 a 2 dias)

- [ ] Criar `tapSynth_tests` ligando a biblioteca `tapSynth` e renderizar uma nota pelo processador
- [ ] Clonar o JUCE 8.0.9 localmente e compilar com `-DTAPSYNTH_JUCE_DIR`
- [ ] Rodar o Mull em um teste simples e confirmar mutantes em `Source/` e nas classes JUCE em cabeçalhos
- [ ] Rodar o pluginval no VST3 compilado e escolher o nível de rigor

Fase 1: testes (3 a 4 dias)

- [ ] Funções de renderização e análise em `tests/common/`
- [ ] Testes de sanidade, incluindo a variante com patches aleatórios
- [ ] Testes espectrais (inclusive em 44,1, 48 e 96 kHz) e temporais
- [ ] Gravar as referências do golden master (configurações padrão e extremas)
- [ ] Fixar tolerâncias e a grade de tolerâncias em `tests/common/Tolerances.h`
- [ ] Criar a tag `tests-frozen`

Fase 2: defeitos (em paralelo com a fase 1, pelo colega)

- [ ] Levantar 10 a 15 correções reais em plugins JUCE open source, cobrindo todas as categorias
- [ ] Transformar cada uma em patch, com cabeçalho de identificação
- [ ] Configurar o escopo do Mull (projeto e JUCE)
- [ ] Preparar as alterações inofensivas

Fase 3: execução (2 a 3 dias)

- [ ] Escrever `run_experiment.py` e o workflow `mutation.yml`
- [ ] Rodar defeitos, mutantes e alterações inofensivas; gerar o `results.csv`
- [ ] Rodar a varredura de tolerâncias

Fase 4: análise e escrita

- [ ] Tabelas, gráfico da curva de tolerância e testes estatísticos
- [ ] Resultados, discussão e guia de recomendação
- [ ] Preparar a demonstração da apresentação: defeito de taxa de amostragem tocado ao vivo e o teste espectral falhando na CI

Comparação de frameworks (secundária)

O experimento usa só o GoogleTest. A comparação com Catch2 e doctest é uma análise de recursos, ilustrada com um teste representativo ("toda nota tem a altura certa") portado para cada framework.

Necessidade do teste de áudio	GoogleTest	Catch2	doctest
Comparação aproximada	`EXPECT_NEAR` (absoluta), `EXPECT_FLOAT_EQ` (baseada em ULP)	`WithinAbs`, `WithinRel`, `WithinULP`	`Approx` (`epsilon` relativo, mais `scale`)
Mesmo teste para vários valores	`TEST_P` + `INSTANTIATE_TEST_SUITE_P`	`GENERATE`	Laços, `SUBCASE`, casos de teste com template
Verificações de áudio próprias	Macros `MATCHER` do gMock	Classes de matcher próprias	Sem sistema de matchers; usar funções auxiliares
Contexto extra quando falha	`SCOPED_TRACE`	`INFO`, `CAPTURE`	`INFO`, `CAPTURE`
Relatório legível pela CI	XML via `--gtest_output`	Reporter JUnit	Reporter JUnit

Ponto relevante para áudio: a tolerância do `Approx` do doctest é relativa, então ±0,5% vale 0,55 Hz em 110 Hz e 8,8 Hz em 1760 Hz, enquanto ±2 Hz absoluto vale o mesmo em toda a faixa.

Pendências do repositório

Resolver antes de tornar o repositório o pacote público de replicação:

- [ ] Verificar a licença do repositório original do tutorial tapSynth (The Audio Programmer) e adicionar um arquivo LICENSE compatível
- [ ] Criar um README que credite o tutorial, o colega e este trabalho, e explique como rodar os testes e o experimento
- [ ] Tirar a chave da API do código-fonte: variável de ambiente ou arquivo de configuração local listado no `.gitignore`
- [ ] Atualizar o CLAUDE.md: remover "There are no tests" e adicionar a seção do estudo (trecho acima)
- [ ] Confirmar onde está o trabalho anterior de `tapsynth_core` e GoogleTest (outra branch ou ainda não começou) e alinhar a proposta

Limitações e questões em aberto

O estudo é um estudo de caso: os resultados valem para sintetizadores subtrativos simples em JUCE e são um primeiro ponto de dados, não regras gerais.

Limitações conhecidas:

- Um único plugin; outros tipos (compressores, reverbs, sintetizadores complexos) podem se comportar diferente.
- Resultados ligados ao JUCE 8.0.9.
- Os defeitos injetados podem não representar todos os defeitos reais; mitigado pelas correções reais e pela mutação automática.
- Um único framework no experimento.
- Viés de autoria; mitigado pelo congelamento dos testes, pelos defeitos criados pelo colega e pela comparação entre origens (QP4).

Questões em aberto:

- [ ] Datas de entrega do artigo e da apresentação
- [ ] Validar tema, perguntas e escopo com o orientador
- [ ] Nível de rigor do pluginval
- [ ] Valores das tolerâncias e da grade de tolerâncias
- [ ] Número de patches aleatórios na variante de sanidade
- [ ] Se o Mull gera mutantes nas classes JUCE em cabeçalhos (resposta na fase 0)

Referências

As três primeiras foram verificadas nas fontes; as demais vieram de memória e precisam ser conferidas antes de entrar no artigo.

Verificadas:

- VALENTAN, A.; WOTAWA, F. On the Automation of Audio Plugin Testing. *QRS 2021*, p. 270–278. IEEE Xplore — trabalho mais próximo; base do desenho com mutação.
- BOGDAN, A.; VERANO MERINO, M.; MALAVOLTA, I. [título exato a copiar da página do artigo; estudo de mineração sobre plugins VST open source]. *MSR 2025*, p. 513–525. dblp — lacuna nas práticas de teste.
- BALTES, S. et al. Guidelines for Empirical Studies in Software Engineering involving Large Language Models. arXiv — base para declarar o uso de IA.

A verificar:

- [ ] BARR, E. T. et al. The Oracle Problem in Software Testing: A Survey. *IEEE TSE*, 2015.

- [ ] JIA, Y.; HARMAN, M. An Analysis and Survey of the Development of Mutation Testing. *IEEE TSE*, 2011.
- [ ] DENISOV, A.; PANKEVICH, S. Mull it over: mutation testing based on LLVM. *ICSTW*, 2018.
- [ ] RUNESON, P.; HÖST, M. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 2009.
- [ ] pluginval (Tracktion), repositório no GitHub, citado como software com data de acesso.